

Apunte de Cátedra

ESTRUCTURAS DE DATOS

**Analista de Sistemas y
Licenciatura en Sistemas**

Equipo de cátedra:
Dra. Sandra I. Casas
Lic. Héctor Reinaga
AdeS. Daniel Gonzalez

**Universidad Nacional de la Patagonia Austral
Unidad Académica Río Gallegos**

Índice

ANALISIS DE ALGORITMOS	1
1.- Introducción	1
2.- Soporte matemático	1
3.- Modelo	3
4.- ¿Qué analizar?	3
5.- Cálculo del tiempo de ejecución	5
5.1.- Un ejemplo sencillo	5
5.2.- Reglas generales	5
5.3.- Soluciones al problema de la suma de la subsecuencia máxima	7
5.4.- Logaritmos en el tiempo de ejecución	11
5.5.- Verificación del análisis	14
ALGORITMOS DE ORDENAMIENTO	16
1.- Introducción	16
2.- Algoritmos básicos de ordenamiento	16
2.1.- Ordenamiento por inserción	16
2.2.- Ordenamiento por selección	18
2.3.- Ordenamiento por intercambio (burbuja)	19
2.4.- Ordenamiento por intercambio (burbuja mejorado)	20
3.- Algoritmos avanzados de ordenamiento	20
3.1.- Ordenamiento por mezcla (mergesort)	20
3.2.- Ordenamiento mediante montículos (heapsort)	21
3.3.- Ordenamiento rápida de Hoare (quicksort)	23
3.4.- Ordenamiento por incrementos (shellsort)	25
3.5.- Ordenamiento por sacudida o vibración (shakersort)	26
4.- Otros algoritmos avanzados de ordenamiento	27
4.1.- Ordenamiento por urnas (binsort)	27
4.2.- Ordenamiento por residuos (radixsort)	27
ALGORITMOS DE BUSQUEDA	29
1.- Introducción	29
2.- Algoritmos básicos de búsqueda	29
2.1.- Búsqueda secuencial	29
2.2.- Búsqueda binaria	30
3.- Algoritmos avanzados de búsqueda	31
3.1.- Búsqueda por interpolación	31
3.2.- Búsqueda Fibonacci	32
4.- Hashing	33
4.1.- Métodos de transformación de claves	34
4.1.1.- Restas sucesivas	34
4.1.2.- Método de división o resto	34
4.1.3.- Método del medio cuadrado	35
4.1.4.- Truncamiento	35
4.1.5.- Método de superposición	35
4.2.- Soluciones al problema de las colisiones	36
4.2.1.- Rehashing o reasignación	36
4.2.1.1.- Prueba lineal o secuencial	37
4.2.1.2.- Prueba cuadrática	39
4.2.1.3.- Doble direccionamiento hash	39
4.2.2.- Arreglos anidados o cubos	40

Índice

4.2.3.- Encadenamiento o tablas hash abiertas	41
4.2.4.- Zona de desbordamiento	42
ARBOLES	43
1.- Introducción	43
2.- Terminología fundamental	43
2.1.- Orden de los nodos	44
2.2.- Recorridos de un árbol	45
2.2.1.- Recorrido en profundidad	45
2.2.2.- Recorrido en anchura	46
2.3.- Árboles etiquetados	47
3.- Árboles binarios	47
3.1.- TDA árbol binario. Definición	47
3.2.- Implementaciones de árboles binarios	48
3.2.1.- Mediante arreglos	48
3.2.2.- Mediante referencias	50
3.3.- Árboles binarios de expresión	50
3.4.- Árboles binarios de búsqueda	51
3.5.- Árboles binarios balanceados	53
3.5.1.- Árboles binarios balanceados AVL	53
3.5.1.1.- Inserción en árboles binarios balanceados AVL	53
3.5.1.2.- Borrado en árboles binarios balanceados AVL	54
3.5.1.3.- Implementación de árboles binarios balanceados AVL	55
4.- Árboles multiamino	59
4.1.- TDA árbol multiamino. Definición	59
4.2.- Implementaciones de árboles multiamino	59
4.2.1.- Mediante arreglos	59
4.2.2.- Mediante listas de hijos	60
4.2.3.- Basada en árboles binarios	61
4.3.- Árboles multiamino B	62
4.3.1.- Búsqueda en árboles multiamino B	63
4.3.2.- Inserción en árboles multiamino B	64
4.3.3.- Borrado en árboles multiamino B	64
4.3.4.- Implementación de árboles multiamino B	66
4.4.- Árboles multiamino B ⁺	72
4.4.1.- Búsqueda en árboles multiamino B ⁺	73
4.4.2.- Inserción en árboles multiamino B ⁺	73
4.4.3.- Borrado en árboles multiamino B ⁺	73
4.5.- Árboles multiamino Trie	74
4.5.1.- Representaciones de árboles multiamino Trie	75
4.5.2.- Búsqueda, inserción y borrado de árboles multiamino Trie	76
4.5.3.- Implementación de árboles multiamino Trie	77
5.- Bosques	79
GRAFOS	81
1.- Introducción	81
2.- Terminología fundamental	81
2.1.- Representación gráfica de grafos	81
2.2.- Definiciones básicas en grafos dirigidos	82

Índice

3.- TDA. Grafo	86
4.- Implementaciones de grafos	86
4.1.- Mediante matrices de adyacencia	86
4.2.- Mediante listas de adyacencia	88
5.- Operaciones sobre grafos	90
5.1.- Recorridos	90
5.1.1.- Recorrido en profundidad	90
5.1.2.- Recorrido en anchura	92
5.2.- Algoritmos de caminos mínimos	93
5.2.1.- Algoritmo de Dijkstra	94
5.2.2.- Algoritmo de Floyd-Warshall	96
5.2.3.- Algoritmo de Bellman-Ford	97
5.2.4.- Algoritmo de Ford-Fulkerson	98
5.2.5.- Algoritmo de Kruskal y Prim	98
5.3.- Algoritmo de Fleury (recorridos eulerianos)	98
REFERENCIAS	100

Apunte de Cátedra

ANALISIS DE ALGORITMOS

1.- INTRODUCCION

Un algoritmo es un conjunto de instrucciones sencillas, claramente especificado, que se debe seguir para resolver un problema. Una vez que se da un algoritmo para un problema y se decide que es correcto, un paso importante es determinar la cantidad de recursos, como tiempo y espacio, que requerirá. Un algoritmo que resuelve un problema, pero tarde un año en hacerlo, difícilmente será de utilidad. De manera similar, un algoritmo que necesita un gigabyte de memoria principal no es útil.

Es necesario estudiar:

- Como calcular el tiempo que emplea un programa.
- Como reducir el tiempo de ejecución de un programa.
- Los resultados del uso indiscriminado de la recursión.

2.- SOPORTE MATEMATICO

En general, el análisis requerido para estimar el uso de recursos de un algoritmo es una cuestión teórica, y por lo tanto, necesita un marco formal. Comenzamos con algunas definiciones matemáticas:

Las siguientes cuatro definiciones serán válidas en todo nuestro estudio:

Definición: $T(n) = O(f(n))$ si existen constantes c y n_0 tales que $T(n) \leq c \cdot f(n)$ cuando $n \geq n_0$

Definición: $T(n) = \Omega(g(n))$ si existen constantes c y n_0 tales que $T(n) \leq c \cdot g(n)$ cuando $n \geq n_0$

Definición: $T(n) = \Theta(h(n))$ si y solo si $T(n) = O(h(n))$ y $T(n) = \Omega(h(n))$.

Definición: $T(n) = O(p(n))$ si $T(n) = O(p(n))$ y $T(n) \neq \Theta(p(n))$

El objetivo de estas definiciones es establecer un orden relativo entre funciones. Dadas dos funciones, por lo general hay puntos donde una función es menor que la otra, de modo que no tienen sentido afirmar que, por ejemplo, $f(n) < g(n)$. Así, se comparan sus *tasas de crecimiento relativas*. Cuando esto se aplica al análisis de algoritmos, se verá por qué ésta es la medida importante.

Aunque $1000n$ es mayor que n^2 para valores pequeños de n , n^2 crece con una tasa mayor, y así, n^2 finalmente será la función mayor. El punto de cambio es, en este caso, $n = 1000$. La primera definición dice que finalmente existe un punto n_0 pasado el cual $c \cdot f(n)$ es siempre al menos tan grande como $T(n)$, de tal modo que se ignoran los factores constantes, $f(n)$ es al menos tan grande como $T(n)$. En este caso, se tiene $T(n) = 1000n$, $f(n) = n^2$, $n_0 = 1000$ y $c = 1$. Se podría usar $n_0 = 10$ y $c = 100$. Así se puede decir que $1000n = O(n^2)$ (orden n cuadrada). Esta notación se conoce como *O grande*. Con frecuencia, en vez de decir "orden ...", suele decirse "O grande..".

Si se usan los operadores tradicionales de desigualdad para comparar las tasas de crecimiento, entonces la primera definición dice que la tasa de crecimiento de $T(n)$ es menor o igual ($\leq$) que la de $f(n)$. La segunda definición, $T(n) = \Omega(g(n))$ (dígase "omega"), dice que la tasa de crecimiento de $T(n)$ es mayor o igual ($\geq$) que la de $g(n)$. La tercera definición, $T(n) = \Theta(h(n))$ (dígase "theta"), dice que la tasa de crecimiento de $T(n)$ es igual ($=$) a la de $h(n)$. La última definición, $T(n) = o(p(n))$ (dígase "o pequeña"), dice que la tasa de crecimiento de $T(n)$ es menor ($<$) que la tasa de crecimiento de $p(n)$. Esta es diferente de la *O grande*, porque *O grande* permite que las tasas de crecimiento se igualen.

Para demostrar que alguna función $T(n) = O(f(n))$, no se suelen aplicar estas definiciones formalmente, sino que se usa un repertorio de resultados conocidos. En general, esto significa que una demostración es un proceso muy sencillo y no debe implicar cálculo, excepto en circunstancias extraordinarias.

Cuando se dice que $T(n) = O(f(n))$, se está garantizando que la función $T(n)$ crece a una velocidad no mayor que $f(n)$; así $f(n)$ es una *cota superior* de $T(n)$. Como esto implica que $f(n) = \Omega(T(n))$, se dice que $T(n)$ es una *cota inferior* de $f(n)$.

Por ejemplo, n^3 crece más rápido que n^2 , así se puede decir que $n^2 = O(n^3)$ o $n^3 = \Omega(n^2)$. $f(n) = n^2$ y $g(n) = 2n^2$ crecen a la misma velocidad, así que ambas, $f(n) = O(g(n))$ y $f(n) = \Omega(g(n))$, se cumplen. Cuando dos funciones crecen a la misma velocidad, la decisión de representar esto o no con $\Theta()$ puede depender del contexto particular. Intuitivamente, si $g(n) = 2n$, entonces $g(n) = O(n^4)$, $g(n) = O(n^3)$ y $g(n) = O(n^2)$ son

Apunte de Cátedra

técnicamente correctas, pero es obvio que la ultima opción es la mejor respuesta. Escribir $g(n) = \Theta(n^2)$ dice no solo que $g(n) = O(n^2)$, sino también que el resultado es tan bueno (exacto) como es posible.

Las cosas importantes a saber son:

Regla 1:

Si $T_1(n) = O(f(n))$ y $T_2(n) = O(g(n))$, entonces

- (a) $T_1(n) + T_2(n) = \max(O(f(n)), O(g(n)))$
- (b) $T_1(n) * T_2(n) = O(f(n) * g(n))$

Regla 2:

Si $T(x)$ es un polinomio de grado n , entonces $T(x) = \Theta(x^n)$

Regla 3:

$\log^k n = O(n)$ para cualquier k constante. Esto indica que los logaritmos crecen muy lentamente.

Para ver que la regla 1(a) es correcta, nótese que por definición existen cuatro constantes c_1 , c_2 , n_1 y n_2 , tales que $T_1(n) \leq c_1 f(n)$ para $n \geq n_1$ y $T_2(n) \leq c_2 g(n)$ para $n \geq n_2$. Sea $n_0 = \max(n_1, n_2)$. Entonces, para $n \geq n_0$, $T_1(n) \leq c_1 f(n)$ y $T_2(n) \leq c_2 g(n)$, de modo que $T_1(n) + T_2(n) \leq c_1 f(n) + c_2 g(n)$. Sea $c_3 = \max(c_1, c_2)$. Entonces,

$$\begin{aligned} T_1(n) + T_2(n) &\leq c_3 f(n) + c_3 g(n) \\ &\leq c_3 (f(n) + g(n)) \\ &\leq 2c_3 \max(f(n), g(n)) \\ &\leq c \max(f(n), g(n)) \end{aligned}$$

para $c = 2c_3$ y $n \geq n_0$

Y así podrían demostrarse las otras relaciones dadas anteriormente. Esta información es suficiente para ordenar por tasas de crecimiento la mayoría de las funciones comunes:

<i>Función</i>	<i>Nombre</i>
c	constante
$\log n$	logarítmica
$\log^2 n$	logarítmica cuadrada
n	lineal
$n \log n$	
n^2	cuadrática
n^3	cúbica
2^n	exponencial

Hay varios puntos que destacar. Primero, es muy mal estilo incluir constantes o términos de orden menor en una O grande. No se debe decir $T(n) = O(2n^2)$ o $T(n) = O(n^2 + n)$. En ambos casos, la forma correcta es $T(n) = O(n^2)$. Esto significa que en cualquier análisis que requiera una respuesta O grande, todos los tipos de simplificaciones son posibles. Por lo regular pueden ignorarse los términos de menor orden y desecharse las constantes. La precisión que se requiere en estos casos es considerablemente menor.

En segundo lugar, siempre se pueden determinar las tasas de crecimiento relativo de dos funciones $f(n)$ y $g(n)$ mediante el cálculo $\lim_{n \rightarrow \infty} f(n) / g(n)$, usando la regla de L'Hôpital¹ si es necesario.

El límite puede tener uno de cuatro valores:

- El límite es 0: esto significa que $f(n) = o(g(n))$.
- El límite es $c \neq 0$: esto significa que $f(n) = \Theta(g(n))$.
- El límite es ∞ : esto significa que $g(n) = o(f(n))$.
- El límite oscila: no hay ninguna relación.

¹ La regla de L'Hôpital establece que si $\lim_{n \rightarrow \infty} f(n) = \infty$ y $\lim_{n \rightarrow \infty} g(n) = \infty$, entonces $\lim_{n \rightarrow \infty} f(n) / g(n) = \lim_{n \rightarrow \infty} f'(n) / g'(n)$ donde $f'(n)$ y $g'(n)$ son las derivadas de $f(n)$ y $g(n)$, respectivamente.

Apunte de Cátedra

Casi nunca es necesaria la utilización de este método. En general la relación entre $f(n)$ y $g(n)$ se puede obtener por simple álgebra. Por ejemplo, si $f(n) = n \log n$ y $g(n) = n^{1.5}$, entonces para decidir cual de las funciones $f(n)$ y $g(n)$ crece con mayor rapidez, lo que realmente se necesita es determinar que crece más rápidamente, $\log n$ o $n^{1.5}$. Esto es como determinar que crece con mayor rapidez, $\log^2 n$ o n . Este problema es sencillo, porque es bien sabido que n crece más rápidamente que cualquier potencia de un logaritmo. Así, $g(n)$ crece con mayor rapidez que $f(n)$.

Una nota de estilo: es incorrecto decir $f(n) \leq O(f(n))$ porque la desigualdad está implícita en la definición. Es erróneo escribir $f(n) \geq O(g(n))$, pues no tiene sentido.

3.- MODELO

Para analizar algoritmos en un marco formal, se necesita un modelo de computación. Nuestro modelo es básicamente una computadora normal, en la cual las instrucciones se ejecutan de modo secuencial. El modelo tiene el repertorio estándar de instrucciones sencillas, como adición, multiplicación, comparación y asignación, pero a diferencia de las computadoras reales, ésta tarda exactamente una unidad de tiempo en hacer cualquier cosa (sencilla). Para ser razonable, se supondrá que, como una computadora moderna, este modelo tiene enteros de tamaño fijo (digamos 32 bits) y que no tiene instrucciones refinadas, como la inversión de matrices o la clasificación, que claramente no se pueden hacer en una unidad de tiempo. También suponemos una memoria infinita.

Es obvio que este modelo tiene algunas debilidades. En la vida real, por supuesto, no todas las operaciones tardan exactamente el mismo tiempo. En particular, en este modelo una lectura de disco cuenta igual que una adición, aun cuando la adición suele ser varios órdenes de magnitud más rápida. También, al suponer memoria infinita, nunca hay que preocuparse por faltas de página, lo cual puede ser un problema real, en especial en algoritmos eficientes. Este puede ser un grave problema en muchas aplicaciones.

4.- ¿QUÉ ANALIZAR?

En general, el recurso más importante a analizar es el tiempo de ejecución. Varios factores afectan el tiempo de ejecución de un programa. Algunos, como el compilador y la computadora usada, están mas allá del alcance de cualquier modelo teórico, así que, aunque son importantes, no hemos de tratarlos. Los otros factores relevantes son el algoritmo usado y su entrada.

El tamaño de la entrada suele ser la consideración principal. Se definen dos funciones, $T_{\text{prom}}(n)$ y $T_{\text{peor}}(n)$, como el tiempo de ejecución promedio y el del peor caso, respectivamente, empleados por un algoritmo para una entrada de tamaño n . Claro esta, $T_{\text{prom}}(n) \leq T_{\text{peor}}(n)$. Si hay más de una entrada, esas funciones pueden tener más de un argumento.

Cabe señalar que en general la cantidad requerida es el tiempo del peor caso, a menos que se especifique otra cosa. Una razón para esto es que da una cota para todas las entradas, incluyendo entradas particularmente malas, que un análisis del caso promedio no puede ofrecer. La otra razón es que la cota del caso promedio suele ser mucho más difícil de calcular. En algunos casos, la definición de “promedio” puede afectar el resultado. (Por ejemplo, ¿qué es una entrada promedio para el problema siguiente?).

Como ejemplo, se considerará el siguiente problema:

PROBLEMA DE LA SUMA DE LA SUBSECUENCIA MÁXIMA

Dados enteros (posiblemente negativos) $a_1, a_2, \dots, a_n$, encontrar el valor máximo de $\sum_{k=i}^j a_k$ (Por conveniencia, la suma de la subsecuencia máxima es 0 si todos los enteros son negativos).

Ejemplo: Para la entrada $-2, 11, -4, 13, -5, -2$, la respuesta es 20 (de a_2 hasta a_4).

Este problema es interesante sobre todo porque existen muchos algoritmos para resolverlo, y su rendimiento varía drásticamente. Estudiaremos cuatro algoritmos para resolver el problema. En la siguiente tabla se muestra el tiempo de ejecución en cierta computadora (el modelo exacto carece de importancia) para estos algoritmos.

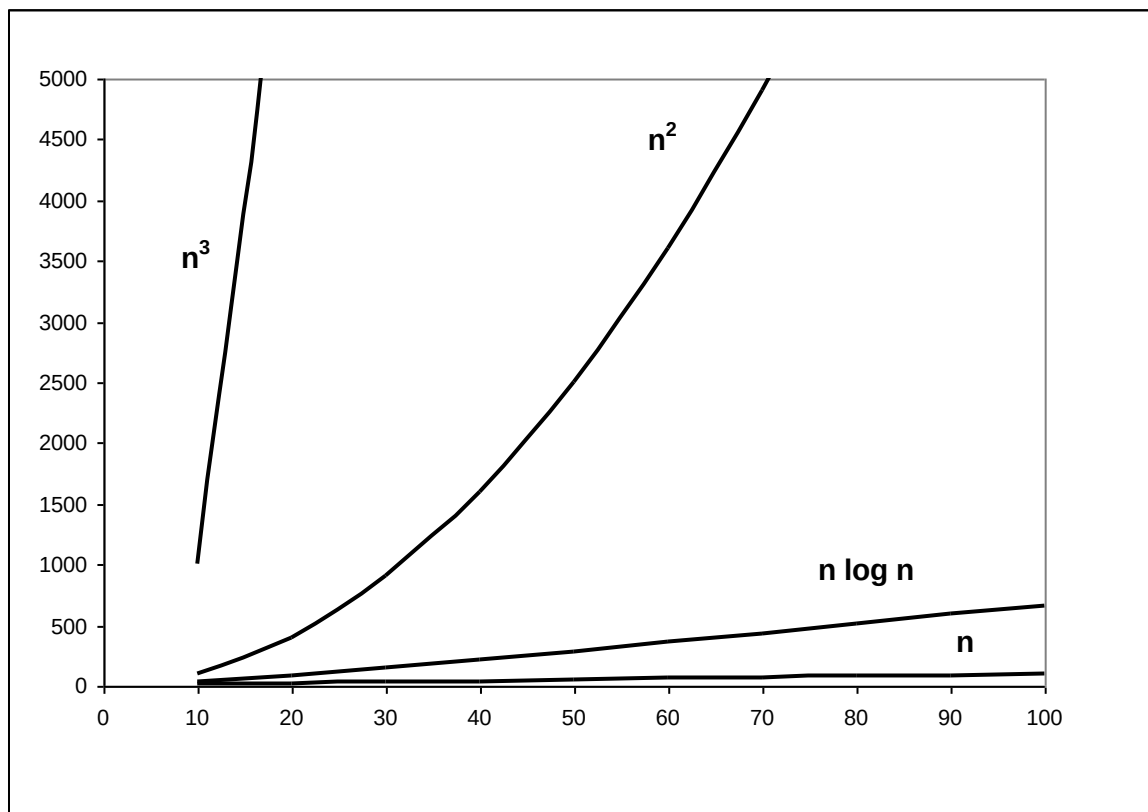
Apunte de Cátedra

Algoritmo		1	2	3	4
Tiempo		$O(n^3)$	$O(n^2)$	$O(n \log n)$	$O(n)$
Tamaño de la Entrada	n = 10	0.00103	0.00045	0.00066	0.00034
	n = 100	0.47015	0.01112	0.00486	0.00063
	n = 1000	448.77	1.1233	0.05843	0.00333
	n = 10000	NA	111.13	0.68631	0.03042
	n = 100000	NA	NA	8.0113	0.29832

Hay varias cosas importantes que observar en esta tabla. Para una entrada pequeña, todos los algoritmos se ejecutan en un instante, así que si solo se espera una pequeña cantidad de datos, no valdría la pena esforzarse por tratar de encontrar un algoritmo muy ingenioso. Por otro lado, existe un gran mercado hoy en día de reescritura de programas que fueron creados cinco años atrás o más, basados en la suposición, invalida hoy, de que la entrada es pequeña. Esos programas son demasiados lentos ahora porque usaron algoritmos deficientes. Para entradas grandes, el algoritmo 4 es claramente la mejor opción (aunque el 3 también es utilizable)

Segundo, los tiempos dados no incluyen el tiempo requerido para leer la entrada. Para el algoritmo 4, el tiempo de lectura de la entrada desde un disco es probablemente de un orden de magnitud mayor que el requerido para resolver el problema. Esto es característico en muchos algoritmos eficientes. La lectura de datos suele ser el cuello de botella; una vez que se leen los datos, el problema se puede resolver rápidamente. Para algoritmos ineficientes esto no es cierto, y hay que valerse de recursos de cómputo significativo. Así, es importante que, siempre que sea posible, los algoritmos sean suficientemente eficientes para no volverse el cuello de botella del problema.

La siguiente figura muestra las tasas de crecimiento de los tiempos de ejecución de los cuatro algoritmos. Aun cuando esta gráfica solo presenta valores para n entre 10 y 100, las tasas de crecimiento relativas son evidentes. Aunque el gráfico del algoritmo 3 parece lineal, es fácil verificar que no lo es usando una regla (o una hoja de papel).



Apunte de Cátedra

Si representáramos gráficamente el rendimiento para valores mayores, las diferencias serían realmente significativas. Lo cual sería una ilustración espectacular de cuan inútiles son los algoritmos ineficientes, incluso para cantidades moderadamente grandes de datos.

5.- CALCULO DEL TIEMPO DE EJECUCION

Hay varias formas de calcular el tiempo de ejecución de un programa. La tabla anterior se obtuvo empíricamente. Si se espera que dos programas tomen tiempos parecidos, es probable que la mejor forma de decidir cual es más rápido sea codificarlos y ejecutarlos.

En general existen varias ideas algorítmicas, y es deseable la eliminación rápida de las ineficaces, por lo que se suele requerir un análisis. Más aún, la habilidad de hacer un análisis permite aprender a diseñar algoritmos eficientes. El análisis también suele detectar cuellos de botella, que vale la pena codificar con gran cuidado.

Para simplificar el análisis, adoptaremos el convenio de que no hay unidades de tiempo particulares. Así, se desechan las constantes iniciales. Tampoco se toman en cuenta los términos de orden menor. Como O grande es una cota superior, nunca se debe subestimar el tiempo de ejecución de un programa. En efecto, la respuesta obtenida es una garantía de que el programa termina en cierto lapso; el programa puede terminar antes de este, pero nunca después.

5.1.- Un ejemplo sencillo

Aquí está un fragmento de programa sencillo para calcular $\sum_{i=1}^n i^3$

```
int suma(int n)
{
    int i, sumaParcial;
{1}    sumaParcial=0;
{2}    for(i=1; i<=n; i++)
{3}        sumaParcial=sumaParcial + i * i * i;
{4}    return sumaParcial;
}
```

El análisis de este algoritmo es sencillo. Las declaraciones no cuentan en el tiempo. Las líneas {1} y {4} cuentan por una unidad de tiempo cada una. La línea {3} cuenta por cuatro unidades cada vez que se ejecuta (dos multiplicaciones, una adición y una asignación) y se ejecuta n veces, para un total de $4n$ unidades. La línea {2} tiene el costo oculto de la inicialización, $n + 1$ para todas las comprobaciones y n para todos los incrementos, lo cual da $2n + 2$. Se ignoran los costos de llamar y retornar de la función, para un total de $6n + 4$. Así, se dice que la función es de orden $O(n)$.

Si tuviéramos que efectuar este trabajo cada vez que necesitemos analizar un programa, pronto la tarea se haría irrealizable. Por fortuna, como estamos dando la respuesta en términos de O grande, hay muchos medios de abreviar que se pueden seguir sin afectar la respuesta final. Por ejemplo, la línea {3} obviamente es un enunciado $O(1)$ (por ejecución), así que es inútil contar precisamente si lleva dos, tres o cuatro unidades; esto no importa. La línea {1} es obviamente insignificante comparada con el ciclo for, así que es inútil consumir tiempo aquí. Esto lleva a varias reglas generales obvias.

5.2.- Reglas Generales

Regla 1 - Ciclos FOR

El tiempo de ejecución de un ciclo for es a lo sumo el tiempo de ejecución de las instrucciones que están en el interior del ciclo for (incluyendo las condiciones) por el número de iteraciones.

Apunte de Cátedra

Regla 2 - Ciclos *FOR ANIDADOS*

Analizarlos de adentro hacia fuera. El tiempo de ejecución total de una proposición dentro del grupo for anidados es el tiempo de ejecución de la proposición multiplicado por el producto de los tamaños de todos los ciclos for.

Como ejemplo, el siguiente fragmento de programa es $O(n^2)$:

```
for(i=1; i<=n; i++)  
    for(j=1; j<=n; j++)  
        k=k + 1;
```

Regla 3 - Proposiciones consecutivas

Simplemente se suman (lo cual significa que el máximo es el único que cuenta).

Como ejemplo, el siguiente fragmento de programa, que tiene trabajo $O(n)$ seguido de trabajo $O(n^2)$:

```
for(i=0; i<n; i++)  
    a[i]=0;  
for(i=0; i<n; i++)  
    for(j=1; j<=n; j++)  
        a[i]=a[i] + a[i] + i + j;
```

Regla 4 - *IF/ELSE*

Para el fragmento

```
if(cond)  
    s1  
else  
    s2
```

El tiempo de ejecución de una proposición if/else nunca es más grande que el tiempo de ejecución de la condición más el mayor de los tiempos de ejecución de S1 y S2.

Claramente, esto puede ser un tiempo sobrevalorado en algunos casos, pero nunca es una subestimación.

Otras reglas son obvias, pero una estrategia básica de análisis que funciona es ir del interior (o parte más profunda) hacia fuera. Si hay llamadas a funciones, obviamente éstas deben ser analizadas primero. Si hay procedimientos recursivos, hay varias opciones. Si la recursión es un ciclo for ligeramente disfrazado, el análisis suele ser trivial. Por ejemplo, la siguiente función es en realidad solo un ciclo sencillo y obviamente es $O(n)$:

```
int factorial(int n)  
{  
    if(n==0 || n==1)  
        return 1;  
    else  
        return n * factorial(n-1);  
}
```

Este ejemplo es realmente un uso ineficiente de la recursión. Cuando la recursión se usa adecuadamente, es difícil convertirla en una estructura iterativa sencilla. En este caso, el análisis implicará una relación de recurrencia que hay que resolver. Para ver que puede suceder, considérese el siguiente programa, que resulta ser un uso horrible de la recursión:

Apunte de Cátedra

{Cálculo de los números de Fibonacci}
{Supóngase $n \geq 0$ }

```
int fibonacci(int n)
{
{1}    if(n==0 || n==1)
{2}        return n;
        else
{3}        return fibonacci(n-1) + fibonacci(n-2);
}
```

A primera vista, éste parece ser un uso muy inteligente de la recursión. Sin embargo, si el programa se codifica para valores de n alrededor de 30, se hace evidente que el programa es terriblemente ineficiente. El análisis es bastante sencillo. Sea $T(n)$ el tiempo de ejecución de la función $\text{fibonacci}(n)$. Si $n = 0$ o $n = 1$, entonces el tiempo de ejecución es algún valor constante, que es el tiempo requerido para evaluar la condición de la línea {1} y regresar. Se puede decir que $T(0) = T(1) = 1$, ya que las constantes no importan. El tiempo de ejecución para otros valores de n se mide entonces en relación con el tiempo de ejecución del caso base. Para $n > 2$, el tiempo de ejecución de la función es el trabajo constante de la línea {1} más el trabajo de la línea {3}. La línea {3} consta de una adición más dos llamadas a función. Puesto que las llamadas a función no son operaciones simples, se debe analizar por separado. La primera llamada a función es $\text{fibonacci}(n-1)$ y, por lo tanto, por la definición de T , requiere $T(n-1)$ unidades de tiempo. Un razonamiento similar muestra que la segunda llamada requiere $T(n-2)$ unidades de tiempo. Entonces el tiempo total requerido es $T(n-1) + T(n-2) + 2$, donde 2 cuenta por el trabajo de la línea {1} mas la adición de la línea {3}. Así, para $n \geq 2$, se tiene la siguiente fórmula del tiempo de ejecución de $\text{fibonacci}(n)$:

$$T(n) = T(n-1) + T(n-2) + 2$$

Como $\text{fibonacci}(n)$ es igual a $\text{fibonacci}(n-1) + \text{fibonacci}(n-2)$, es fácil demostrar por inducción que $T(n) \geq \text{fibonacci}(n)$ y así el tiempo de ejecución de este programa crece *exponencialmente*. Esto es lo peor posible. Con un arreglo simple y un ciclo for, se obtiene una reducción sustancial del tiempo de ejecución. Este programa es lento porque se efectúa una cantidad enorme de trabajo redundante, violándose la cuarta regla de la recursión². Nótese que la primer llamada de la línea {3}, $\text{fibonacci}(n-1)$, realmente calcula $\text{fibonacci}(n-2)$ en algún momento. Esta información es desechada y se compone recursivamente y hace que el tiempo de ejecución sea enorme. Este es tal vez el mejor ejemplar de la máxima “no calcular nada mas de una vez” y no debe ahuyentar al programador del uso de la recursión. Existen usos sobresalientes de la recursión.

5.3.- Soluciones al problema de la suma de la subsecuencia máxima

Ahora se presentan los cuatro algoritmos que resuelven el problema de la suma de la subsecuencia máxima antes planteado. (punto 4)

```
int sumaSubsecuenciaMaxima(int[] a, int n)
{
    int estaSuma, sumaMax, mejorI, mejorJ, i, j, k;
{1}    sumaMax=0; mejorI=0; mejorJ=0;
{2}    for(i=0; i<n; i++)
{3}        for(j=0; j<n; j++)
        {
{4}            estaSuma=0;

{5}            for(k=i; k<=j; k++)
```

² Regla del interés compuesto. El trabajo nunca se debe duplicar resolviendo el mismo ejemplar de un problema en llamadas recursivas separadas.

Apunte de Cátedra

```

{6}          estaSuma=estaSuma + a[k];
{7}          if(estaSuma > sumaMax)
              { //actualiza sumaMax, mejorI, mejorJ
{8}          sumaMax=estaSuma;
{9}          mejorI=i;
{10}         mejorJ=j;
              }
            }
{11}         return sumaMax;
            }

```

Algoritmo 1.

Este algoritmo funciona. El tiempo de ejecución es $O(n^3)$ y se debe por completo a las líneas {5} y {6}, las cuales consisten en una proposición $O(1)$ inmersa en tres ciclos for anidados. El ciclo de la línea {2} es de tamaño n . El segundo ciclo tiene un tamaño $n - i + 1$, el cual podría ser pequeño, pero también puede ser de tamaño n . Se puede suponer lo peor, con el entendimiento de que esta cota final puede ser un poco alta. El tercer ciclo tiene un tamaño $j - i + 1$, que de nuevo, se debe suponer de tamaño n . El total es $O(1 \cdot n \cdot n \cdot n) = O(n^3)$. La proposición {1} solo toma $O(1)$ en total, y las proposiciones {7} a {10} toman sólo $O(n^2)$ en total, puesto que son proposiciones sencillas dentro de dos iteraciones.

Se puede evitar el tiempo de ejecución cúbico eliminando un ciclo for. Es obvio que esto no siempre es posible, pero en este caso hay una gran cantidad de cálculos innecesarios en el algoritmo. La ineficiencia que el algoritmo mejorado corrige sobre los cálculos de las líneas {5} y {6} del algoritmo 2 es indebidamente costosa. A continuación se muestra el algoritmo mejorado. Claramente este algoritmo es $O(n^2)$; el análisis es aun más sencillo.

```

int sumaSubsecuenciaMaxima(int[] a, int n)
{
    int estaSuma, sumaMax, mejorI, mejorJ, i, j;
{1}    sumaMax=0; mejorI=0; mejorJ=0;
{2}    for(i=0; i<n; i++)
        {
{3}        estaSuma=0;
{4}        for(j=i; j<n; j++)
            {
{5}                estaSuma=estaSuma + a[j];
{6}                if(estaSuma > sumaMax)
                    { //actualiza sumaMax, mejorI, mejorJ
                      sumaMax=estaSuma;
                      mejorI=i;
                      mejorJ=j;
                    }
            }
        }
{7}    return sumaMax;
}

```

Algoritmo 2.

Este problema tiene solución $O(n \log n)$ recursiva y relativamente compleja, que ahora describiremos. Si no fuera porque existe una solución (lineal) $O(n)$, este sería un excelente ejemplo del poder de la recursión. El algoritmo se vale de la estrategia “divide y vencerás”. La idea es partir el problema en dos subproblemas más o menos iguales, cada uno de los cuales es de tamaño igual a la mitad del original. Entonces los

Apunte de Cátedra

subproblemas se resuelven recursivamente. Esta es la parte de “dividir”. La etapa de “vencer” consiste en unir las dos soluciones de los subproblemas, y posiblemente en hacer un poco de trabajo adicional, para llegar a una solución del problema global.

En este caso, la suma de la subsecuencia máxima puede estar en uno de tres lugares. O esta entera en la primera mitad de la entrada, o en la segunda mitad, o bien pasa por el punto medio y se encuentra en ambas mitades. Los primeros dos casos se pueden resolver recursivamente. El último caso se obtiene, encontrando la suma mayor en la primera mitad que incluya al último elemento de esa primera mitad y la suma más grande de la segunda mitad que incluya al primer elemento de la segunda mitad. Estas dos sumas se pueden sumar. Como ejemplo, considérese la siguiente entrada:

Primera mitad				Segunda mitad			
4	-3	5	-2	-1	2	6	-2

La suma de la subsecuencia máxima de la primera mitad es 6 (los elementos entre a_1 y a_3), y para la segunda mitad es 8 (los elementos entre a_6 y a_7).

La suma máxima de la primera mitad incluye al último elemento de la primera mitad es 4 (elementos entre a_1 y a_4), y la suma máxima de la segunda mitad que incluye al primer elemento de la segunda mitad es 7 (elementos entre a_5 y a_7). Así, la suma máxima que abarca ambas mitades y pasa por el medio es $4 + 7 = 11$ (elementos entre a_1 y a_7).

Se ve, entonces, que de las tres formas de encontrar la subsecuencia de longitud máxima, para el ejemplo, la mejor forma es incluir elementos de ambas mitades. Así la respuesta es 11.

El código del algoritmo 3 merece algún comentario. La forma general de la llamada al procedimiento recursivo es pasar el arreglo de entrada junto con los límites izquierdo y derecho, que delimitan la porción del arreglo de entrada sobre la que se va a operar. Un programa manejador de una línea hace esto pasando los bordes 1 y n para evitar hacer una copia. Si se olvida hacer esto el tiempo de ejecución de la rutina puede variar drásticamente, como se verá después.

Las líneas {1} a {4} manejan el caso base. Si $izq == der$, entonces hay un elemento, y este es la subsecuencia máxima si el elemento es no negativo. El caso $izq > der$ no es posible a menos que n sea negativo (aunque perturbaciones menores en el código podrían echar a perder esto). Las líneas {6} y {7} hacen dos llamadas recursivas. Se puede ver que las llamadas recursivas son siempre sobre problemas más pequeños que el original, aunque, de nuevo, perturbaciones menores en el código pueden destruir esta propiedad. Las líneas {8} a la {12} y después de la {13} a la {17} calculan las dos sumas máximas que alcanzan el centro.

Claro está, el algoritmo 3 requiere más esfuerzo de codificación que cualquiera de los dos anteriores. Sin embargo, la brevedad de un código no siempre implica que este sea el mejor. Como se vio en la tabla anterior, donde se muestran los tiempos de ejecución de los algoritmos, este algoritmo es considerablemente más rápido que los otros dos, excepto con entradas de tamaño reducido.

```

int sumaSubsecuenciaMaxima(int[] a, int n)
{
    return sumaSubsecuenciaMaxima(a, 0, n-1);
}

int sumaSubsecuenciaMaxima(int[] a, int izq, int der)
{
    int sumaMaxIzq, sumaMaxDer, sumaMaxIzqBorde, sumaMaxDerBorde;
    int sumaBordeIzq=0, sumaBordeDer=0, centro, i;
    if(izq == der) //caso base

```

Apunte de Cátedra

```

{2}      if(a[izq] > 0)
{3}      return a[izq];
          else
{4}      return 0;
          else
          {
{5}      centro=(izq + der) / 2;
{6}      sumaMaxIzq= sumaSubsecuenciaMaxima(a, izq, centro);
{7}      sumaMaxDer= sumaSubsecuenciaMaxima(a, centro + 1, der);
{8}      sumaMaxIzqBorde=0;
{9}      for(i=centro; i>=izq; i--)
          {
{10}     sumaBordeIzq=sumaBordeIzq + a[i];
{11}     if(sumaBordeIzq > sumaMaxIzqBorde)
{12}     sumaMaxIzqBorde=sumaBordeIzq;
          }
{13}     sumaMaxDerBorde=0;
{14}     for(i=centro+1; i<=der; i++)
          {
{15}     sumaBordeDer=sumaBordeDer + a[i];
{16}     if(sumaBordeDer > sumaMaxDerBorde)
{17}     sumaMaxDerBorde=sumaBordeDer;
          }
{18}     return Math.max(Math.max(sumaMaxIzq, sumaMaxDer),
                          sumaMaxIzqBorde + sumaMaxDerBorde);
          }
      }

```

Algoritmo 3.

El tiempo de ejecución se analiza casi en la misma forma que el programa que calcula los números de Fibonacci. Sea $T(n)$ el tiempo que lleva resolver un problema de suma de la subsecuencia máxima de tamaño n . Si $n = 1$, entonces el programa usa una cantidad de tiempo constante para ejecutar la línea {1} a {4}, a la cual se tomará como la unidad. Así $T(1) = 1$. De otro modo, el programa debe realizar dos llamadas recursivas, los dos ciclos que están entre las líneas {5} y {18}. Los dos ciclos for se combinan para alcanzar todo elemento entre a_1 y a_n , con un trabajo constante en el interior de los ciclos, así que el tiempo consumido en las líneas {9} a {17} es $O(n)$. El código de las líneas {1} a {5}, {8} y {18} es constante en trabajo y se puede ignorar cuando se compara con $O(n)$. El resto del trabajo se realiza en las líneas {6} y {7}. Esas líneas resuelven dos problemas de suma de la subsecuencia máxima con tamaño $n/2$ (suponiendo n par). Así, estas líneas utilizan $T(n/2)$ unidades de tiempo cada una, para un total de $2T(n/2)$. El tiempo total consumido por el algoritmo, por lo tanto, es $2T(n/2) + O(n)$. Esto da las ecuaciones

$$T(1) = 1$$

$$T(n) = 2T(n/2) + O(n)$$

Para simplificar los cálculos, se puede sustituir por n el término $O(n)$ de la ecuación anterior; como de todos modos $T(n)$ se expresará en notación O grande, esto no afectará la respuesta. Si $T(n) = 2T(n/2) + n$, y $T(1) = 1$, entonces $T(2) = 4 = 2 * 2$, $T(4) = 12 = 4 * 3$, $T(8) = 32 = 8 * 4$, $T(16) = 80 = 16 * 5$. El patrón que es evidente, y que se puede obtener, es que si $n = 2^k$, entonces $T(n) = n * (k+1) = n \log n + n = O(n \log n)$.

Este análisis supone que n es par, ya que de otra forma $n/2$ no está definido. Por la naturaleza recursiva del análisis, esto es realmente válido solo cuando n es una potencia de 2, pues de otra forma tarde o temprano se

Apunte de Cátedra

Llega a un subproblema que no es de tamaño par, y la ecuación es inválida. Cuando n no es una potencia de 2, se requiere un análisis un poco más complejo, pero el resultado O grande permanece sin cambio. Se calcula el número total de llamadas recursivas que efectúa el algoritmo. Sea $R(n)$ el número de llamadas recursivas hechas para resolver un problema de subsecuencia de tamaño n . Entonces $R(1) = 0$, ya que éste es el caso base. El número de llamadas recursivas de la línea {6} es igual al número de llamadas hechas cuando se resuelve el primer subproblema más, por supuesto, la llamada recursiva real. La misma lógica se aplica a la línea {7}. Esta dice que $R(n) = 2R(n/2)$. Esto se puede resolver (cuando n es una potencia de 2) para llegar a $R(n) = 2n - 2$. El cuarto algoritmo para encontrar la suma de la subsecuencia máxima, es más simple de implantar que el algoritmo recursivo y también es más eficiente.

```
int sumaSubsecuenciaMaxima(int[] a, int n)
{
    int estaSuma, sumaMax, mejorI, mejorJ, i, j;
    i=0; estaSuma=0; sumaMax=0; mejorI=0; mejorJ=0;
    for(j=0; j<n; j++)
    {
        estaSuma=estaSuma + a[j];
        if(estaSuma > sumaMax)
        { //actualiza sumaMax, mejorI, mejorJ
            sumaMax=estaSuma;
            mejorI=i;
            mejorJ=j;
        }
        else
            if(estaSuma <= 0)
            {
                i=j+1;
                estaSuma=0;
            }
    }
    return sumaMax;
}
```

Algoritmo 4.

Debe quedar claro por que la cota del tiempo es correcta, pero lleva más tiempo entender por que de hecho funciona. Esto se deja al estudiante. Una ventaja adicional del algoritmo es que solo recorre una vez los datos, y una vez que se lee y procesa $a[i]$, no necesita recordarse esto. Así, si el arreglo está en disco o cinta, se puede leer secuencialmente, sin necesidad de almacenar parte alguna en la memoria principal. Más aún, en cualquier momento, el algoritmo puede dar correctamente una respuesta al problema de la subsecuencia para los datos que ya ha leído (los otros algoritmos no comparten esta propiedad). Los algoritmos que pueden hacer esto se conocen como *algoritmos en línea*. Un algoritmo en línea que requiere solo espacio constante y se ejecuta en tiempo lineal es el mejor posible.

5.4.- Logaritmos en el tiempo de ejecución

Es posible que el aspecto más confuso del análisis de algoritmos se centre en el logaritmo. Ya se ha visto que algunos algoritmos “divide y vencerás” se ejecutarán en un tiempo $O(n \log n)$. Además de los algoritmos de “divide y vencerás”, la aparición más frecuente de los logaritmos está alrededor de la siguiente regla general: *un algoritmo es $O(\log n)$ si usa un tiempo constante ($O(1)$) en dividir el problema en partes (normalmente en $\frac{1}{2}$)*. Por otro lado, si se requiere un tiempo constante simplemente para reducir el problema en una cantidad constante (como hacer el problema más pequeño en 1), el algoritmo es $O(n)$.

Apunte de Cátedra

Algo que debe ser obvio es que solo esa clase especial de problemas puede ser $O(\log n)$. Por ejemplo si la entrada es una lista de n números, un algoritmo solo debe tardar $\Omega(n)$ en leer los datos. Así, cuando se habla de algoritmos $O(\log n)$ para esa clase de problemas, por lo regular se supone que la entrada fue leída antes. A continuación se dan tres ejemplos de comportamiento logarítmico.

BUSQUEDA BINARIA

El primer ejemplo se conoce como búsqueda binaria. Dado un entero x y enteros $a_1, a_2, \dots, a_n$, los cuales están ordenados y en memoria, encontrar i tal que $a_i = x$, o devolver $i = 0$ si x no esta en la entrada.

La solución obvia consiste en rastrear la lista de izquierda a derecha y se ejecuta en tiempo lineal. No obstante, este algoritmo no aprovecha el hecho que la lista esta ordenada, y por lo tanto, probablemente no sea la mejor solución. La mejor estrategia es probar si x esta en la mitad de la lista, de ser así, la respuesta esta a la mano. Si x es menor que el elemento del centro, se puede aplicar la misma estrategia al subarreglo ordenado a la izquierda del elemento central; si x es mayor que el elemento del centro se buscará en la mitad derecha. (También esta el caso de cuando parar). A continuación se muestra el código para la búsqueda binaria (la respuesta es el contenido de medio, en este caso la variable pos).

```
int busquedaBinaria(int[] a, int x, int n)
{
    int primero, medio, ultimo, pos=-1;
{1}    primero=0;
{2}    ultimo=n-1;
{3}    while(primer<=ultimo & pos== -1)
    {
{3}        medio=(primero + ultimo) / 2;
{5}        if(a[medio] == x)
{6}            pos=medio;
        else
{7}            if(a[medio] < x)
{8}                primero=medio + 1;
            else
{9}                ultimo=medio - 1;
    }
{10}    return pos;
}
```

Por supuesto todo el trabajo realizado por iteración dentro del ciclo es $O(1)$, así que el análisis requiere determinar el número de veces que se itera. El ciclo empieza con $\text{ultimo} - \text{primero} = n - 1$ y termina cuando $\text{ultimo} - \text{primero} \leq 0$. Cada vez que se itera el valor $\text{ultimo} - \text{primero}$ debe reducirse al menos a la mitad de su valor original; así, el número de veces que se itera es a lo sumo $\lceil \log(n-1)+2 \rceil$. (Por ejemplo, si $\text{ultimo} - \text{primero} = 128$, entonces los valores máximos de $\text{ultimo} - \text{primero}$ después de cada iteración son 64, 32, 16, 8, 4, 2, 1, 0. Así, el tiempo de ejecución es $O(\log n)$. En forma equivalente, se podría escribir una fórmula recursiva para el tiempo de ejecución, pero esta clase de enfoque por fuerza bruta es innecesario cuando se entiende que esta ocurriendo realmente y por qué.

La búsqueda binaria se puede ver como una estructura de datos. Permite la operación buscar en tiempo $O(\log n)$, pero todas las demás operaciones (en particular insertar) requieren tiempo $O(n)$. En aplicaciones donde los datos son estáticos (esto es, no se permiten inserciones y eliminaciones), puede ser una estructura de datos muy útil. La entrada podría ser ordenada solo una vez, y después los accesos serían muy rápidos. Un ejemplo podría ser un programa que necesita mantener información acerca de la tabla periódica de los elementos (problema típico en física y química). Esta tabla es relativamente estable, dado que es poco frecuente que se agreguen nuevos elementos. Los nombres de los elementos se podrían almacenar

Apunte de Cátedra

ordenados. Como solo hay unos 110 elementos, a lo sumo se requerirían ocho accesos para encontrar un elemento. Realizar la búsqueda secuencial requeriría muchos más accesos.

ALGORITMO DE EUCLIDES

Un segundo ejemplo es el algoritmo de Euclides para calcular el máximo común divisor. El máximo común divisor de dos enteros es el mayor entero que divide a ambos. Así, $\text{maximoComunDivisor}(50,15) = 5$. El siguiente algoritmo calcula $\text{maximoComunDivisor}(m, n)$, suponiendo que $m \geq n$. (Si $m < n$, la primera iteración del ciclo los intercambia)

```
int maximoComunDivisor(int m, int n)
{
    int resto;
    while(n > 0)
    {
        resto=m % n;
        m=n;
        n=resto;
    }
    return m;
}
```

El algoritmo funciona a base de calcular continuamente los restos hasta llegar a 0. La respuesta es el último distinto de cero. Así, si $m = 1989$ y $n = 159$, entonces la secuencia de restos es 399, 393, 6, 3, 0. Por lo tanto, $\text{maximoComunDivisor}(1989,1590) = 3$. Como lo muestra el ejemplo, este es un algoritmo rápido.

Como antes, el tiempo de ejecución total del algoritmo depende de lo grande que sea la secuencia de residuos. Aunque $\log n$ parece ser una buena respuesta, no es en absoluto obvio que el valor del resto tenga que descender en un factor constante, pues se ve que el resto va de 399 a solo 393 en el ejemplo. En efecto, el resto no disminuye en un factor constante en una iteración. Sin embargo, se puede demostrar que, después de dos iteraciones, el resto es a lo sumo la mitad de su valor original. Esto podría demostrar que el número de iteraciones es a lo sumo $2 \log n = O(\log n)$ y establecer el tiempo de ejecución. Esta demostración es fácil, por lo que se incluye aquí. Se infiere directamente del siguiente teorema.

Teorema: Si $m > n$, entonces $m \% n < m / 2$.

Demostración: Hay dos casos. Si $n \leq m / 2$, entonces obviamente, como el resto es menor que n , el teorema se cumple para este caso. El otro caso es $n > m / 2$. Pero entonces n cabe en m una vez con un resto $m - n < m / 2$, demostrando el teorema.

Uno puede preguntarse si esta es la mejor cota posible, ya que $2 \log n$ es cerca de 20 para este ejemplo, y solo se realizaron siete operaciones. Resulta que la constante se puede mejorar ligeramente, para llegar a $1.44 \log n$, en el peor caso (el cual es alcanzable si m y n son números de Fibonacci consecutivos). El rendimiento en el caso promedio del algoritmo de Euclides requiere páginas y páginas de análisis matemático altamente complicado, y resulta finalmente que el número medio de iteraciones es de cerca de $(12 \ln 2 \ln) / \pi^2 + 1.47$

Apunte de Cátedra

EXPONENCIACION

El último ejemplo trata de la elevación de un entero a una potencia (que también es entera). Los números que resultan de la exponenciación suelen ser bastantes grandes, así que un análisis sólo sirve si se supone que se tienen una máquina con capacidad para almacenar tales enteros grandes (o un compilador que los pueda simular). Contaremos el número de multiplicaciones como la medida del tiempo de ejecución.

El algoritmo obvio para calcular x^n usa $n - 1$ multiplicaciones. El algoritmo recursivo siguiente lo hace mejor. Las líneas {1} a la {4} manejan el caso base de la recursión. De otra forma, si n es par, se tienen $x^n = x^{n/2} \cdot x^{n/2}$, y si n es impar, $x^n = x^{(n-1)/2} \cdot x^{(n-1)/2} \cdot x$.

```

int potencia(int x, int n)
{
{1}    if(n == 0)
{2}        return 1;
    else
{3}        if(n == 1)
{4}            return x;
    else
{5}        if(n % 2 == 0)
{6}            return potencia(x * x, n / 2);
    else
{7}        return potencia(x * x, n / 2) * x;
}

```

Por ejemplo, para calcular x^{62} el algoritmo hace los siguientes cálculos, en los que intervienen solo nueve multiplicaciones:

$$x^3 = (x^2) x, \quad x^7 = (x^3)^2 x, \quad x^{15} = (x^7)^2 x, \quad x^{31} = (x^{15})^2 x, \quad x^{62} = (x^{31})^2$$

Claramente, el número de multiplicaciones requeridas es a lo sumo $2 \log n$, porque a lo sumo se necesitan dos multiplicaciones (n es impar) para partir en dos, el problema. De nuevo, se puede escribir y resolver una fórmula recursiva. La simple intuición hace obvia la necesidad de un enfoque por la fuerza bruta.

A veces es interesante ver cuánto código se puede abreviar sin afectar la corrección. En el código escrito, de hecho, {3} a {4} son innecesarias ya que si $n = 1$, entonces la línea {7} resuelve ese caso. También la línea {7} se puede escribirse como:

```

{7}    return potencia(x, n - 1) * x;

```

sin afectar la corrección del programa. En efecto, el programa seguirá ejecutándose en $O(\log n)$, porque la secuencia de multiplicaciones es la misma que antes. Sin embargo, todas las alternativas a la línea {6} que se encuentran a continuación no sirven, aunque parezcan correctas:

```

{6a}    return potencia(potencia(x, 2), n / 2);
{6b}    return potencia(potencia(x, n / 2), 2);
{6c}    return potencia(x, n / 2) * potencia(x, n / 2);

```

Las líneas {6a} y {6b} son incorrectas porque cuando $n = 2$, una de las llamadas recursivas a potencia tiene 2 como segundo argumento. Así, no se obtiene progreso, y resulta un ciclo infinito.

Usar la línea {6c} afecta la eficiencia porque hay dos llamadas recursivas de tamaño $n / 2$ en vez de una. Un análisis demostrará que ese tiempo de ejecución ya no será $O(\log n)$.

Apunte de Cátedra

5.5.- Verificación del análisis

Una vez que se ha realizado un análisis, es deseable ver si la respuesta es correcta y es la mejor posible. Una forma de hacer esto es codificar el programa y ver si los tiempos de ejecución observados empíricamente concuerdan con los predichos por el análisis. Cuando n se duplica, el tiempo de ejecución crece en un factor de 2 para los programas lineales, 4 para programas cuadráticos y 8 para programas cúbicos. Los programas que se ejecutan en tiempo logarítmicos sólo se incrementan en una cantidad constante cuando n se duplica, y los programas que se ejecutaban en $O(n \log n)$ tardan ligeramente más del doble bajo las mismas circunstancias. Estos incrementos pueden ser difíciles de determinar si los términos de orden menor tienen coeficientes relativamente grandes y n no es bastante grande. Un ejemplo es el salto de $n = 10$ a $n = 100$ en el tiempo de ejecución de las diferentes implementaciones del problema de la suma de la subsecuencia máxima. También puede ser muy difícil diferenciar los programas lineales de los programas $O(n \log n)$ a partir de una simple comprobación empírica.

Otro truco que se usa mucho para verificar que algún programa es $O(f(n))$ consiste en calcular los valores de $T(n) / f(n)$ para un intervalo de n (por lo regular espaciado en factores de 2), donde $T(n)$ es el tiempo de ejecución empírica observado. Si $f(n)$ es una respuesta cercana al tiempo de ejecución, entonces los valores calculados convergen a una constante positiva. Si $f(n)$ es una sobreestimación, los valores convergen a cero. Si $f(n)$ está subestimada y, por lo tanto, es incorrecta, los valores divergen.

Por ejemplo, el fragmento de programa de la siguiente figura calcula la probabilidad de que dos enteros positivos distintos, menores o iguales que n y escogidos al azar, sean primos relativos. (Cuando n aumente, la respuesta se acerca a $6/\pi^2$)

```
rel=0;
tot=0;
for(i=1; i<=n; i++)
    for(j=1; j<=n; j++)
    {
        tot=tot + 1;
        if(maximoComunDivisor(i, j) == 1)
            rel=rel + 1;
    }
System.out.println("El porcentaje de pares primos relativos es "+ (double)rel / (double)tot);
```

n	Tiempo de CPU (T)	T/n^2	T/n^3	$T/n^2 \log n$
100	022	.002200	.000022000	.0004777
200	056	.001400	.000007.000	.0002642
300	118	.001311	.000004370	.0002299
400	207	.001294	.000003234	.0002159
500	318	.001272	.000002544	.0002047
600	466	.001294	.000002157	.0002024
700	644	.001314	.000001877	.0002006
800	846	.001322	.000001652	.0001977
900	1086	.001341	.000001490	.0001971
1000	1362	.001362	.000001362	.0001972
1500	3240	.001440	.000000960	.0001969
2000	5949	.001482	.000000740	.0001947
4000	25720	.001608	.000000402	.0001938

La tabla muestra el tiempo de ejecución real observado para esta rutina en una computadora real. La última columna es más probable, y así el análisis obtenido debió ser correcto. Nótese que no hay gran diferencia entre $O(n^2)$ y $O(n^2 \log n)$, ya que los logaritmos crecen muy lentamente.